Model Guide: Staged Transfer-Learning GNN for SJCET Forecasting

Owners: Model track (Vaishakh + 1 member) · **Contracts:** `data-contracts.md`

This guide takes you from nothing to a trained GNN that forecasts next-day weather at SJCET. It covers getting ERA5 and NOAA data, building the graph, the 3-stage transfer-learning training, baselines, the data-efficiency ablation, and exporting results for the frontend.

Each step ends with a **Check**. Don't skip them, because silent data bugs are the main way weather ML projects fail.

---

0. The plan in one page

Research question: does pretraining on large-scale weather data reduce how much local SJCET data is needed for a good forecast?

Forecast task (v1): every day `t`, using the last **K = 7 days**, predict for days `t+1 … t+H` (start with **H = 1**, extend to 3):

- `tmax` (°C), `tmin` (°C), `rain` (mm, trained on `log1p`), and `P(rain ≥ 2.5 mm)`.

One graph for all stages (a "stretched grid", as in Nipen et al., *Regional Data-Driven Weather Modeling with a Global Stretched Grid*, in `About/Papers/`):

Node set	Where	Resolution	Count (approx.)
ERA5 coarse	India box 6–37°N, 68–98°E (outside the Kerala box)	1.0°	~970
ERA5 fine	Kerala/W. Ghats box 8–13°N, 74–78°E	0.25°	~357
GSOD stations	real stations inside the Kerala box	point	~5–15
SJCET	9.727°N, 76.727°E *(verify on a map)*	point	1

The **same model weights** run on the same graph in every stage. Only the data and **which nodes the loss is computed on** change:

Stage	Trains on	Loss on	Learns
1. Pretrain	ERA5, 1990 → train-end	all ERA5 nodes	large-scale weather dynamics
2. Regional	ERA5 + GSOD obs	Kerala ERA5 nodes + GSOD nodes	Western Ghats behaviour, reanalysis → real-observation bias
3. Local	+ SJCET register	SJCET node (others with small weight)	SJCET micro-climate

Ablation (the key result): stage 3 with 10/25/50/100% of SJCET training data, for (a) GNN from scratch, (b) stage 1→3, (c) stage 1→2→3, plus baselines. Plot RMSE vs data fraction.

Work split

	2A: Data & Graph	2B: Model & Training
Steps	1–7, 9 (baselines)	8, 10–14

	2A: Data & Graph	2B: Model & Training
Owns	`model/data/`, `model/baselines.py`	`model/gnn.py`, `train.py`, `evaluate.py`, `predict.py`, `export.py`
Hands over	`processed/*.npz`, `graph.pt`, `dataset.py`	checkpoints, `metrics.json`, `forecast` JSON

2B doesn't have to wait: **day 1, 2A writes a fake `dataset.py`** that returns random tensors with the right shapes (§7). 2B builds the model and training loop against it.

---

1. Environment (both, ½ day)

```
git checkout -b model
mkdir model && cd model
uv init --bare
# CPU torch, same index trick as digitization/pyproject.toml
uv add "torch>=2.4" torch_geometric numpy pandas pyarrow xarray netCDF4 cdsapi \
       scikit-learn xgboost matplotlib pyyaml tqdm
```

Copy the `[[tool.uv.index]]` / `[tool.uv.sources]` block from `digitization/pyproject.toml` so torch comes from the CPU wheel index. `torch_geometric`'s basic layers (`GATv2Conv`, `SAGEConv`) need **no** extra compiled extensions.

Folder layout:

```
model/
├── config.yaml            boxes, years, variables, splits, hyperparameters
├── data/
│   ├── download_era5.py   download_static.py
│   ├── build_era5.py      build_gsod.py   build_sjcet.py
│   ├── build_graph.py     dataset.py
├── baselines.py  gnn.py  train.py  evaluate.py  predict.py  export.py
├── raw/        (gitignored) downloaded NetCDF/CSV
├── processed/  (gitignored) arrays ready for training
└── outputs/    checkpoints/, metrics.json, graph.json, forecasts/
```

Put **every** number (boxes, years, K, H, thresholds) in `config.yaml`, never hard-coded.

GPU: the laptop has no CUDA. Everything here runs on CPU (the graph is small), but for many ablation runs use **Kaggle Notebooks** (free GPU, 30 h/week). Upload `processed/` as a private Kaggle Dataset.

---

2. Decide the time splits *first* (both, 1 hour)

All stages must hold out the **same** validation and test years. Otherwise the model sees the test-period weather at neighbouring nodes during pretraining, which is leakage.

The digitized SJCET record is expected to cover at least Jul 2010–Dec 2016 (scans), plus whatever years the photos add. Start with:

```
splits:
  train: [1990-01-01, 2013-12-31]   # stage 1 & 2 use all of this; stage 3 uses SJCET days in it
  val:   [2014-01-01, 2014-12-31]
  test:  [2015-01-01, 2016-12-31]
```

Update once the digitization track confirms the year coverage. Keep test = the last ~2 years of the SJCET record. **Never use random day-level splits.**

---

3. Acquire ERA5 (2A, 1–2 days, mostly waiting)

3.1 Why CDS and not Open-Meteo

Open-Meteo's free archive API counts every 2 weeks × 10 variables of a location as one call, with a 10,000 calls/day limit. That's fine for one point (SJCET sanity checks) but far too little for ~1,300 nodes × 36 years. Use the **Copernicus Climate Data Store (CDS)**, which gives gridded NetCDF.

3.2 Account

1. Register at https://cds.climate.copernicus.eu.
2. Open the dataset pages **"ERA5 post-processed daily statistics on single levels"** (`derived-era5-single-levels-daily-statistics`) and **"ERA5 hourly data on single levels"** (`reanalysis-era5-single-levels`), and **accept the licence** on each (bottom of the download tab). Missing this is the #1 cause of `403` errors.
3. Put your token in `~/.cdsapirc`: `url: https://cds.climate.copernicus.eu/api key: <your personal access token>`

3.3 What to download

Area `[N, W, S, E] = [37, 68, 6, 98]`, years **1990–2025**, time zone `utc+05:00` (closest to IST).

Variable	Daily statistic	→ our variable
`2m_temperature`	`daily_maximum`	`tmax`
`2m_temperature`	`daily_minimum`	`tmin`
`2m_temperature`	`daily_mean`	`tmean`
`2m_dewpoint_temperature`	`daily_mean`	`td`
`total_precipitation`	`daily_sum`	`rain` (see units below)
`mean_sea_level_pressure`	`daily_mean`	`mslp`
`10m_u_component_of_wind`	`daily_mean`	`u10`
`10m_v_component_of_wind`	`daily_mean`	`v10`
`total_cloud_cover`	`daily_mean`	`tcc`

Static fields (once, from the same daily dataset, any single day): `land_sea_mask`, `geopotential` (÷ 9.80665 → elevation in m).

`model/data/download_era5.py`:

```
import itertools, pathlib, cdsapi

OUT = pathlib.Path("raw/era5"); OUT.mkdir(parents=True, exist_ok=True)
JOBS = [("2m_temperature", "daily_maximum"), ("2m_temperature", "daily_minimum"),
        ("2m_temperature", "daily_mean"), ("2m_dewpoint_temperature", "daily_mean"),
        ("total_precipitation", "daily_sum"), ("mean_sea_level_pressure", "daily_mean"),
        ("10m_u_component_of_wind", "daily_mean"), ("10m_v_component_of_wind", "daily_mean"),
        ("total_cloud_cover", "daily_mean")]
c = cdsapi.Client()
for (var, stat), year in itertools.product(JOBS, range(1990, 2026)):
    f = OUT / f"{var}_{stat}_{year}.nc"
    if f.exists():
        continue                                  # safe to re-run after failures
    c.retrieve("derived-era5-single-levels-daily-statistics", {
        "product_type": "reanalysis", "variable": [var], "daily_statistic": stat,
        "year": str(year), "month": [f"{m:02d}" for m in range(1, 13)],
        "day": [f"{d:02d}" for d in range(1, 32)],
        "time_zone": "utc+05:00", "frequency": "1_hourly",
        "area": [37, 68, 6, 98],
    }, str(f))
```

The ready-made version of this is `model/data/download_era5.py` (reads `model/config.yaml`, 4 parallel requests, resumable). That's 324 requests (~20 MB each, ~7 GB total). CDS queues them, so expect **hours to a day**. Run it in the background (`nohup uv run python data/download_era5.py &`). To halve the wait, each of you runs half the years with your own CDS account.

Units: temperatures in K (−273.15 → °C); pressure in Pa (÷100 → hPa); `total_precipitation` daily sum is metres per day → **× 1000 = mm/day**. *Verify this on one point against Open-Meteo (§3.4) before trusting it.*

3.4 Check

- Open one file with `xarray`, plot the July mean `tmax` map. The Western Ghats should be cooler than the plains, and the monsoon rain should show on the west coast in June–Sep.
- For the grid point nearest SJCET, pull the same days from Open-Meteo (`https://archive-api.open-meteo.com/v1/archive?latitude=9.727&longitude=76.727&start_date=2012-01-01&end_date=2012-12-31&daily=temperature_2m_max,precipit` `Kolkata&models=era5`) and confirm they match to within ~0.5 °C and similar rain totals.

---

4. Build the ERA5 node arrays (2A, 1 day)

`model/data/build_era5.py`, processing **one year at a time** (the laptop has ~7.5 GB RAM):

1. Load all variables for the year, convert units, and stack to `[time, lat, lon, var]`.
2. **Fine nodes:** crop the Kerala box 8–13°N, 74–78°E at native 0.25°.
3. **Coarse nodes:** `ds.coarsen(latitude=4, longitude=4, boundary="trim").mean()` → 1°, then **drop** coarse cells whose centre falls inside the Kerala box (the fine nodes cover it).
4. Flatten to `[time, node, var]` (coarse nodes first, then fine) and append to the yearly list.
5. Concatenate the years and save `processed/era5.npz` with `X` (float32, `[T, N, V]`), `dates`, plus `processed/nodes.parquet` (`node_id, kind, lat, lon, elev, lsm`).

Store rain as **mm** here. The `log1p` transform happens in normalisation (§7).

Check: shape ≈ `[13149, ~1330, 9]`, no NaNs in ERA5, and the plotted time series at the node nearest SJCET looks seasonal.

---

5. Build the GSOD station nodes (2A, 1 day)

NOAA GSOD is already downloaded in `Dataset/<year>/<USAF><WBAN>.csv` (1929–2025). Get the station registry:

```
curl -O https://www.ncei.noaa.gov/pub/data/noaa/isd-history.csv
```

(For more years later: `https://www.ncei.noaa.gov/data/global-summary-of-the-day/access/<year>/<station>.csv`, or whole-year tarballs under `.../global-summary-of-the-day/archive/<year>.tar.gz`.)

`model/data/build_gsod.py`:

1. Select stations with `CTRY == "IN"` inside the Kerala box (8–13°N, 74–78°E) whose `BEGIN`/`END` overlap 1990–2025.
2. Load each station's daily rows for 1990–2025 and convert:

GSOD col	missing code	convert
`MAX`, `MIN`, `TEMP`	`9999.9`	°F → °C: `(f − 32) × 5/9`
`DEWP`	`9999.9`	°F → °C
`PRCP`	`99.99`	inches × 25.4 → mm
`SLP`	`9999.9`	already hPa
`WDSP`	`999.9`	knots (not used in v1)

1. **`PRCP` flag `I`** means the station did not report precipitation. Treat `0.00 I` as **missing**, not zero. Many Indian GSOD rain values are unreliable, so check the yearly totals.
2. Keep stations with ≥ 50% coverage of 1990–2025. Map to our variables (`tmax, tmin, tmean, td, rain, mslp`), set `u10, v10, tcc` to NaN.
3. Append to `[T, N, V]` as new nodes with `kind = gsod`. NaN = missing.

Check: list the stations kept (name, lat/lon, coverage %). Correlation of each station's `tmax` with its nearest ERA5 node should be > 0.6. Plot both for one year.

---

6. Add the SJCET node (2A, ½ day, once a partial `sjcet_daily.csv` exists)

`model/data/build_sjcet.py` reads `data/sjcet/sjcet_daily.csv` (contracts §1).

Time alignment (important). The register is read at ~8:30 AM:

- `rain_mm` on date D = rain from D-1 08:30 to D 08:30 → mostly belongs to **D-1**
- `tmax_c` read on D = max of **D-1**
- `tmin_c` read on D = min of the night before → belongs to **D**

Don't just trust this. **Test lags 0 and −1** for each variable against the nearest ERA5 node and pick the lag with the higher correlation. Write the chosen lags in `config.yaml`.

SJCET provides `tmax, tmin, rain` only. All other variables are NaN for this node (the mask handles it). Rows with `qc_flag = suspect` → set to NaN for **targets** (keep as inputs).

Before the real file arrives, use a **stand-in**: copy the nearest ERA5 fine node's `tmax, tmin, rain` as a fake SJCET node, with weekends set to NaN. The whole pipeline can be tested end-to-end with this.

Check: the SJCET vs nearest-ERA5 scatter for `tmax`. Expect high correlation and a bias of a few °C (ERA5 is a 25 km average). That bias is exactly what stage 3 learns.

---

7. Normalisation, samples and `dataset.py` (2A, 1 day)

Normalisation

- `rain → log1p(rain)` first.
- Per-variable mean/std from **stage-1 training years, ERA5 nodes only**. Save to `processed/norm.json` and use the **same** stats in every stage and at inference. Changing the stats between stages breaks the transfer.

One sample = one day `t`

For every node `n`:

Part	Shape per node	Content
dynamic inputs	`K × V`	normalised values for days `t–K+1 … t`, NaN → 0
input mask	`K × V`	1 where observed
static	5	lat, lon (scaled), elevation/1000, land-sea mask, (optional) distance to coast
time	2	sin, cos of day-of-year of `t+1`
node kind	3	one-hot `era5 / gsod / sjcet`

So `in_dim = 2·K·V + 10` (= 136 for K = 7, V = 9).

Targets `y[n, H, 4]`: normalised `tmax, tmin, log1p(rain)` + `rain ≥ 2.5 mm` (0/1) for days `t+1…t+H`, with `y_mask` (1 where observed).

Per-node loss weight `w[n]` depends on the stage (§11).

Interface (agree on this on day 1)

```
class WeatherDataset(torch.utils.data.Dataset):
    def __init__(self, stage: int, split: str, local_fraction: float = 1.0, seed: int = 0): ...
    def __len__(self): ...
    def __getitem__(self, i) -> dict:
        # x: [N, in_dim]  y: [N, H, 4]  y_mask: [N, H, 4]  w: [N]  date: str
graph = torch.load("processed/graph.pt")   # dict(edge_index [2,E] long, edge_attr [E, 4], node_kind
[N])
```

For stages 2–3, only days where at least one weighted node has a target are included.

---

8. Build the graph (2A, ½ day)

`model/data/build_graph.py`:

- **Edges:** for every node, connect to its **k = 8 nearest** neighbours (haversine distance), then make them symmetric (add the reverse edges, deduplicate).

- Make sure SJCET and every GSOD node connect to ≥ 4 fine ERA5 nodes, and that fine nodes on the Kerala box border connect to coarse nodes (so large-scale info flows in).
- **Edge features** (4): east offset (km/100), north offset (km/100), distance (km/100), elevation difference (km).
- Save `graph.pt` and `outputs/graph.json` (contracts §6, for the frontend map).

Check: plot nodes and edges on a map (matplotlib scatter + lines). No isolated nodes. Every node's degree is between 8 and ~16.

---

9. Baselines (2A, 1–2 days)

Evaluate all on the SJCET test days with observed targets (same metric code as the GNN, §13):

Baseline	How
Persistence	tomorrow = today (rain: today's rain)
Climatology	day-of-year mean from SJCET training data, smoothed ±15 days
ERA5-nearest	ERA5 value at the nearest node *on the target day*. Not a forecast, but shows the reanalysis bias
XGBoost	features: SJCET last 7 days + nearest ERA5 node last 7 days + doy sin/cos; one model per target
LSTM/GRU local-only	SJCET series only, 2 layers × 64, same K and targets

If the GNN can't beat persistence and climatology, something is wrong. These are the floor.

---

10. The model (2B, 2–3 days)

`model/gnn.py`: encode → message passing (residual) → decode, shared by every node.

```
import torch, torch.nn as nn
from torch_geometric.nn import GATv2Conv

class WeatherGNN(nn.Module):
    def __init__(self, in_dim, edge_dim=4, H=1, hidden=128, layers=4, heads=4, dropout=0.1):
        super().__init__()
        self.H = H
        self.enc = nn.Sequential(nn.Linear(in_dim, hidden), nn.GELU(), nn.Linear(hidden, hidden))
        self.convs = nn.ModuleList(
            GATv2Conv(hidden, hidden // heads, heads=heads, edge_dim=edge_dim) for _ in range(layers))
        self.norms = nn.ModuleList(nn.LayerNorm(hidden) for _ in range(layers))
        self.ffs = nn.ModuleList(
            nn.Sequential(nn.LayerNorm(hidden), nn.Linear(hidden, 2 * hidden), nn.GELU(),
                          nn.Linear(2 * hidden, hidden)) for _ in range(layers))
        self.drop = nn.Dropout(dropout)
        self.dec = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(hidden, H * 4))

    def forward(self, x, edge_index, edge_attr):
        h = self.enc(x)
        for conv, norm, ff in zip(self.convs, self.norms, self.ffs):
            h = h + self.drop(conv(norm(h), edge_index, edge_attr))
            h = h + self.drop(ff(h))
        return self.dec(h).view(-1, self.H, 4)   # [..., 0:3] regression, [..., 3] rain logit
```

Batching days. The graph is the same every day, so stack `B` copies with offset node ids:

```
def batch_graph(edge_index, edge_attr, B, N):
    E = edge_index.size(1)
    offs = (torch.arange(B) * N).repeat_interleave(E)
    return edge_index.repeat(1, B) + offs, edge_attr.repeat(B, 1)
# x: [B, N, F] → x.view(B*N, F)
```

Loss (masked, weighted per node):

```
def loss_fn(pred, y, m, w, rain_pos_weight=2.0):
    # pred, y, m: [B*N, H, 4]; w: [B*N]
    w = w[:, None, None]
    reg = ((pred[..., :3] - y[..., :3]) ** 2 * m[..., :3] * w).sum() / (m[..., :3] *
w).sum().clamp(min=1)
    bce = nn.functional.binary_cross_entropy_with_logits(
        pred[..., 3], y[..., 3], pos_weight=torch.tensor(rain_pos_weight), reduction="none")
    cls = (bce * m[..., 3] * w[..., 0]).sum() / (m[..., 3] * w[..., 0]).sum().clamp(min=1)
    return reg + 0.5 * cls
```

Size: ~0.4 M parameters at `hidden = 128, layers = 4`. On CPU, an epoch over 24 years is a few minutes. Things to try later: `layers` 2/4/6, predicting the *change* from today for temperatures, H = 3.

Check (do this before any real training): overfit **16 days** of fake data to near-zero loss. If it can't, the bug is in the code, not the data.

---

11. Training: the 3 stages (2B, 1 week incl. tuning)

`model/train.py --stage {1,2,3} [--init ckpt] [--fraction 0.25] [--seed 0] [--freeze N]`

	Stage 1	Stage 2	Stage 3
Data	ERA5 nodes (GSOD/SJCET inputs masked)	ERA5 + GSOD	ERA5 + GSOD + SJCET
Loss weight `w`	all ERA5 = 1	Kerala fine = 1, GSOD = 2, coarse = 0.1	SJCET = 1, GSOD = 0.1, others = 0.05
Init	random	stage-1 best	stage-2 (or stage-1) best
LR (AdamW)	1e-3, cosine	3e-4	1e-4 (try 3e-5)
Epochs (early stop on val)	30	20	50, patience 10
Batch (days)	16	16	16

Common settings: weight decay 1e-4, gradient clipping 1.0, seed everything, save `best.pt` by val loss to `outputs/checkpoints/<run_id>/`, plus `config.yaml` and a `log.csv` (epoch, train loss, val loss, val RMSE per target).

Stage 3 options to compare: full fine-tune vs `--freeze 2` (encoder + first 2 conv layers frozen). Keeping small weights on non-SJCET nodes reduces **catastrophic forgetting**.

Checks:

- Stage 1 val RMSE on ERA5 `tmax` should clearly beat persistence (≈ 1–1.5 °C on the fine nodes).
- After stage 3, SJCET val RMSE must beat both persistence and ERA5-nearest.
- Plot the learning curves for every run; overfitting in stage 3 is expected with little data.

---

12. The data-efficiency ablation (both, 3–4 days of runs)

Strategy	Init for stage 3
`gnn-scratch`	random (trained on the stage-3 graph and data only)
`gnn-1-3`	stage 1
`gnn-1-2-3`	stage 1 → 2

× `local_fraction ∈ {0.1, 0.25, 0.5, 1.0}` × `seed ∈ {0, 1, 2}` = **36 stage-3 runs** (+ the same fractions for XGBoost and the LSTM).

Sub-sampling SJCET training data: take **whole contiguous months** chosen at random with the seed (not random days, since neighbouring days leak into each other), until the fraction is reached. Val/test stay fixed.

Script it (`ablation.sh` or a Python loop) and run on Kaggle. Each stage-3 run is small. Write every result as one row to `outputs/ablation.csv`.

Result to report: RMSE vs fraction plot with mean ± std bands, and a sentence like *"with 25% of local data, the pretrained GNN matches the from-scratch GNN trained on 100%"* (if that's what you find; report it honestly either way).

---

13. Evaluation (2B, 1–2 days)

`model/evaluate.py` on **SJCET test days with observed targets**:

- Undo the normalisation; rain = `expm1` of the prediction, clipped at 0.
- `tmax`, `tmin`, `rain`: RMSE, MAE, bias (mean pred − obs), R^2, MAPE (temperatures only, because MAPE explodes for rain near 0).
- Rain occurrence (≥ 2.5 mm, the IMD rainy-day threshold): hit rate, false-alarm ratio, accuracy, Brier score of `rain_prob`.
- Per season (Winter Jan–Feb, Pre-monsoon Mar–May, SW monsoon Jun–Sep, NE monsoon Oct–Dec).
- Plots: predicted vs observed time series for the test period; scatter; error by season.

---

14. Export for the frontend (2B, 1 day)

- `model/export.py` writes `outputs/metrics.json` (contracts §5) from the evaluation results and `ablation.csv`.
- `model/predict.py`: `python def predict(date: str, model_name: str = "gnn-1-2-3") -> dict: """Forecast for SJCET using inputs up to and including `date` (replay mode). Returns the forecast JSON from data-contracts.md §4, with `observed` filled when known."""` Also pre-compute forecasts for all test dates into `outputs/forecasts/<date>.json`, so the frontend works without loading torch.

---

15. Schedule (suggested)

Week	2A: Data & Graph	2B: Model & Training
1	CDS account, start downloads (§3), GSOD (§5), fake `dataset.py`	env, `gnn.py`, training loop on fake data, overfit test
2	`build_era5`, graph, real `dataset.py`, SJCET stand-in, baselines	stage 1 on real ERA5, evaluation code
3	real SJCET data in (§6), XGBoost/LSTM baselines	stages 2–3, tuning
4	ablation runs + plots	ablation runs, export, `predict.py`

For the Phase 1 review: show the data pipeline (maps of the nodes, ERA5 vs GSOD vs SJCET checks), the graph plot, baseline numbers, and stage-1 training curves.

---

16. Pitfalls checklist

- [ ] Same val/test years held out in **all** stages (no leakage via pretraining)
- [ ] Same normalisation stats in all stages and at inference
- [ ] Missing ≠ 0 (NaN + mask everywhere; GSOD `PRCP` flag `I`)
- [ ] SJCET 8:30 AM time alignment checked by lag correlation
- [ ] ERA5 rain units verified against Open-Meteo
- [ ] Metrics computed only where observations exist
- [ ] Ablation subsamples whole months; ≥ 3 seeds; report mean ± std
- [ ] Every run's config + log saved next to its checkpoint
- [ ] Beat persistence and climatology before claiming anything